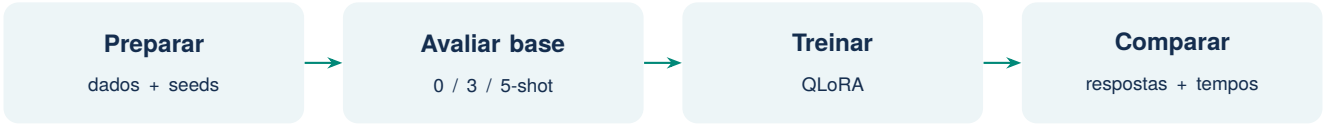
01 | Mapa do projeto

O que o programa faz

Quatro modelos, quatro condições e resultados separados por execução.



O programa compara modelos antes de qualquer ajuste com uma versão ajustada por fine-tuning. Os exemplos de few-shot entram no prompt. Somente a condição **fine-tuned** treina adaptadores.

Bloco A - Quem organiza tudo

rodar_todos.py: escolhe modelos, seeds e repetições; abre um processo por trial e aguarda seu término. Um trial é uma execução completa de um modelo com uma seed e uma repetição.

Bloco B - Quem executa o experimento

modelos/gemma4.py, huatuogpt.py, medgemma.py e qwen35.py: carregam dados e modelo, montam prompts, treinam quando necessário e avaliam as quatro condições. Este guia usa os nomes de funções comuns aos scripts.

Bloco C - Quem prepara e registra

prompts_medicos.py define a persona e lê seeds. **registro.py** grava logs, tabelas e ambiente. **consolidar.py** reúne execuções concluídas. **resumir.py** calcula resumos entre execuções.

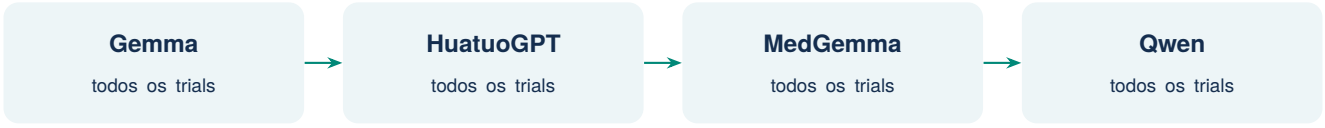
```
CONDITIONS = [('zero-shot', 0, False),
              ('3-shot', 3, False),
              ('5-shot', 5, False),
              ('fine-tuned', 0, True)]
```

Leia cada tupla como: **nome da condição, número de exemplos, deve treinar?** O modelo original é recarregado a cada condição.

02 | Executor

Um modelo por vez

Arquivo: rodar_todos.py | Função principal: main()



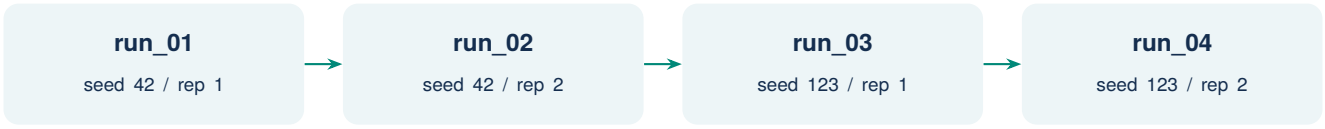
A ordem padrão é gemma4 → huatuogpt → medgemma → qwen35. O executor termina todas as seeds e repetições de um modelo antes de iniciar o seguinte. A opção **–modelos** permite escolher a ordem.

```
for model in args.modelos:
    for seed in args.seeds:
        for repeticao_seed in range(1, args.repeticoes + 1):
            ...
# Depois, cada processo é iniciado e aguardado:
code = process.wait()
```

O trecho abreviado mostra a ordem real dos laços. Não existe execução paralela de LLMs nesse executor. Ao terminar o processo, seus recursos são liberados; isso evita a competição entre os modelos da campanha, mas não impede outros programas de usarem a GPU.

Seed não é a mesma coisa que repetição

–seeds 42 123: muda o sorteio das demonstrações e a aleatoriedade do treino. **–split-seed 42**: mantém a divisão e a seleção dos dados. **–repeticoes 2**: executa duas vezes cada seed, inclusive repetindo o fine-tuning.



Na prática: Esses quatro trials são concluídos para Gemma antes de começar HuatuoGPT. A seed aparece no terminal e em status.json. Nos CSVs consolidados, procure exec_seed.

Se um processo falhar, a campanha para e salva o log. **–retomar** pula apenas trials completos com código e configuração iguais; não continua um treino interrompido.

Treino e teste separados

Bloco de datasets | make_demo_pool()



O script filtra respostas e perguntas, remove perguntas idênticas após normalização, seleciona 240 exemplos e separa 192 para treino e 48 para avaliação. As primeiras 20 perguntas do teste também são usadas no bloco de velocidade.

Cada conjunto tem uma função

MedQuAD: respostas abertas; fornece o treino do fine-tuning, as referências para avaliação textual e as demonstrações de sua própria tarefa.

MedQA: até 100 perguntas do split oficial de teste, com quatro alternativas. As demonstrações vêm do split de treino. Não é usado como treino do fine-tuning neste código.

PubMedQA: até 100 itens de pqa_labeled são reservados para avaliação; o restante fica disponível para demonstrações. É uma divisão própria do experimento, não o teste oficial completo. Também não treina o adaptador.

```
train_dataset, eval_dataset = split_dataset['train'], \
                              split_dataset['test']
# Na condição fine-tuned:
train_regime(model, train_dataset, 5, condition)
```

O que a proteção contra vazamento faz

make_demo_pool() exclui perguntas do teste com texto normalizado igual e remove duplicatas no conjunto de demonstrações. A resposta da pergunta avaliada não é inserida como exemplo.

Na prática: Os exemplos few-shot atuais vêm dos datasets, não de perguntas buscadas na internet. As personas foram adaptadas de referências públicas. A filtragem não verifica paráfrases, sobreposição entre datasets ou dados de pré-treinamento.

04 | Construção do prompt

Onde entram os 3 ou 5 shots

select_demonstrations() → make_messages() → build_prompt()

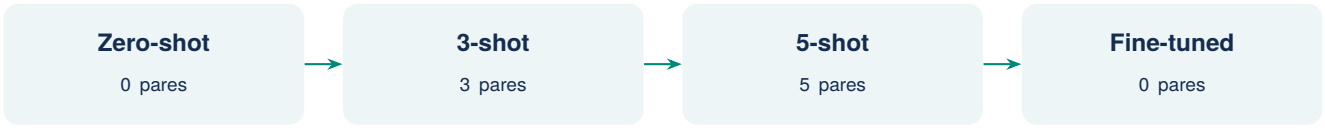


A persona descreve a perspectiva desejada: clínico ou pesquisador biomédico. Ela não conta como shot. Um shot é um par completo de **pergunta e resposta de demonstração**. A persona é mantida entre as condições.

```
messages = [{'role': 'system', 'content': system_prompt}]
for demo in demos:
    messages.extend([
        {'role': 'user', 'content': demo['user']},
        {'role': 'assistant', 'content': demo['assistant']}
    ])
messages.append({'role': 'user', 'content': user_text})
```

Como a seleção permanece reproduzível

A função combina **seed + tarefa + pergunta** em um hash. Esse valor inicializa o sorteio de cinco exemplos. O 3-shot usa os primeiros três dos mesmos cinco usados pelo 5-shot. A pergunta avaliada permanece igual entre as condições.

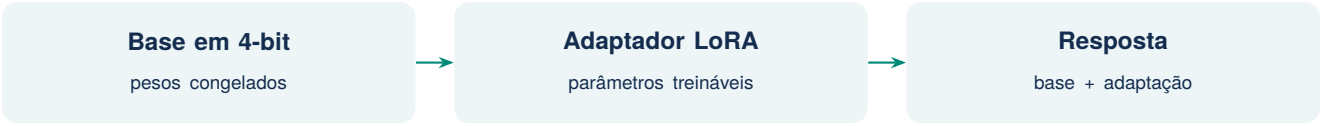


apply_chat_template() transforma as mensagens no formato do modelo. **encode_prompt()** converte o texto em IDs de tokens e reserva espaço para a resposta. Se ultrapassar o limite de contexto configurado, o código gera erro em vez de cortar os exemplos silenciosamente.

Na prática: Para ver o prompt efetivamente montado, abra registros/prompts.csv: messages contém as mensagens, demo_ids identifica os exemplos e seed informa o sorteio. Pedir uma persona não garante correção médica.

O que o QLoRA altera

build_model() e train_regime()



Todos os braços carregam a base com **load_in_4bit=True**. Só o braço fine-tuned adiciona adaptadores LoRA. Treinar adaptadores sobre uma base quantizada é a abordagem QLoRA usada aqui.

```
load_in_4bit=LOAD_IN_4BIT, full_finetuning=False
# Adaptador da condição fine-tuned:
r=8, lora_alpha=16, lora_dropout=0
# Treinamento:
num_train_epochs=5
learning_rate=2e-5
per_device_train_batch_size=2
gradient_accumulation_steps=16
optim="adamw_torch"
```

O rank 8 define o tamanho da adaptação; alpha 16 controla sua escala. São cinco passagens pelos 192 exemplos. O batch efetivo é **2** $\times$ **16 = 32** em uma GPU: acumula gradientes de 16 microbatches antes de atualizar os parâmetros.

Não existe merge explícito neste pacote

O modelo é avaliado com o adaptador ativo e salvo com **model.save_pretrained()**. Não há chamada a merge_and_unload(). Portanto, os tempos incluem a execução da configuração base quantizada + adaptador.

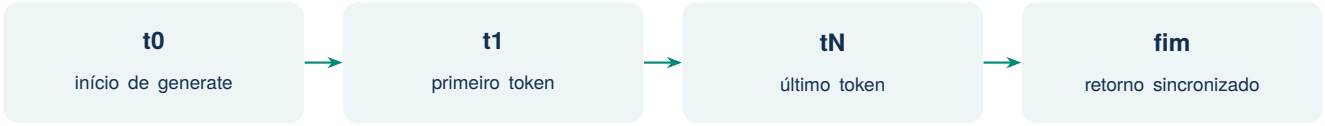
```
Sem merge: y = W x + (alpha/r) B(Ax)
Com merge: W' = W + (alpha/r) BA; y = W' x
```

A segunda linha é apenas uma explicação do conceito, não uma etapa executada. Em bases quantizadas, uma fusão pode envolver arredondamento. O código também não fixa explicitamente NF4 e double quantization; esses detalhes dependem do carregador.

06 | Medição corrigida

Uma geração, um cronômetro

timed_generate(), TokenTimer e timed_answer()



O prompt já foi montado, tokenizado e enviado ao dispositivo quando o cronômetro começa. Há sincronização CUDA antes de começar e depois de generate. O streamer registra cada token observado no host.

```
TTFT = t1 - t0
TTLT = tN - t0
Tempo total = fim - t0
TPOT = (TTLT - TTFT) / (N - 1)
Velocidade de decodificação = 1 / TPOT
```

A correção: foi removida a chamada extra que gerava um token só para estimar TTFT. Agora os tempos pertencem à mesma geração. Decodificar os IDs em texto e limpar a resposta ocorre depois do cronômetro.

Como os tokens são conferidos

O primeiro callback precisa coincidir com os IDs do prompt. Depois, o código exige um token por callback e compara a sequência capturada com os IDs retornados por generate. Se houver diferença, ele para com erro. Esse contrato é para batch 1 e a geração autoregressiva atual.

Resposta curta e encerramento

No teste de velocidade, min_new_tokens = max_new_tokens = 160, equivalente ao tamanho fixo do tutorial. O código rejeita saídas com outra contagem. Na avaliação textual, 160 continua sendo apenas o teto. N conta os IDs efetivamente gerados, inclusive EOS quando emitido. Se N = 1, a taxa de decodificação fica NaN: não existe intervalo entre tokens. finished verifica EOS, inclusive se ocorrer exatamente no limite.

Na prática: No CSV de velocidade, tokens_per_s mede decodificação. No CSV de respostas, tokens_per_s é N/tempo total; decode_tokens_per_s é a taxa sem o primeiro token. Não misture essas colunas.

A equação segue calculate_metrics() do vLLM e do repositório do tutorial: divisor N - 1. Aqui, TTLT é a latência até o último token; total_s também inclui o retorno de generate. O TPOT acima explica o cálculo, mas não é uma nova coluna. A medição continua local, sem HTTP. Tokens de raciocínio também contam.

Como interpretar os resultados

evaluate_model(), evaluate_benchmarks() e save_summaries()



MedQuAD: gera respostas para 48 perguntas. Token F1 mede sobreposição de palavras; ROUGE-L usa a maior subsequência comum. NLL mede a perda da resposta de referência; perplexidade é exp(NLL). Essas métricas não comprovam segurança ou qualidade clínica.

MedQA e PubMedQA: pick_option() faz um forward pass e compara os logits das letras permitidas. Não gera uma justificativa livre. O resultado é accuracy e precision/recall/F1 macro sobre alternativas, não sobre especialidades médicas.

Arquivos que vale abrir primeiro

status.json: modelo, seed, repetição, hash do código e status.
execucao.log: progresso e mensagens de erro.
ambiente.json: versões e configurações; nvidia-smi.txt descreve a GPU.

***-benchmark/speed_per_question.csv:** tempo e tokens por pergunta.
***-benchmark/precision_per_question.csv:** previsão e acerto.
***-extended/answers_per_question.csv:** resposta, referência e métricas.
registros/prompts.csv: mensagens e exemplos usados.

O que a consolidação significa

consolidar.py reúne apenas trials com status ok e acrescenta exec_modelo, exec_repeticao e exec_seed. Resultados de trials incompletos permanecem na pasta individual. resumir.py resume médias e diferenças entre condições pareadas por execução.

Na prática: A velocidade usa 20 perguntas e um warm-up curto. O pacote mostra médias e intervalos, não p95/p99. Repetir a mesma seed não é o mesmo que testar novas perguntas. Os resumos entre execuções agrupam seeds: mantenha exec_seed para análises separadas.

A memória de pico do bloco MedQuAD também inclui o cálculo de NLL. Não a interprete como memória exclusiva da geração.

08 | Uso e referências

Como executar e conferir

Execute os comandos na pasta extraída experimento_llms.

1. Confira a ordem sem carregar modelos

```
python rodar_todos.py --planejar --seeds 42 123 \
  --repeticoes 1
```

2. Execute os mesmos trials, um modelo por vez

```
python rodar_todos.py --seeds 42 123 --repeticoes 1 \
  --saida resultados_corrigidos
```

Os comandos usam continuação de linha de shell Linux; também podem ser escritos em uma linha. Requer ambiente CUDA/NVIDIA com as dependências do pacote. Use uma pasta de saída nova para não misturar medições antigas com as corrigidas.

3. Resuma as execuções concluídas

```
python resumir.py resultados_corrigidos
```

O executor já consolida ao final de cada processo. O padrão sem argumentos é **seed 42 e dez repetições por modelo**: 40 trials e 40 fine-tunings. O exemplo acima reduz para oito trials. A mesma seed não garante identidade numérica em todo hardware.

Referências para conferir os conceitos

1. vLLM - Benchmark CLI: definições e escopo de medição.
2. vLLM - calculate_metrics, em serve.py: referência de cálculo por token.
3. Transformers - streamers.py: interface put/end.
4. Hugging Face PEFT - LoRA: adaptadores e merge.

O texto do tutorial foi fornecido pelo usuário e conferido com seu repositório público. Em async_request_openai_completions(), min_tokens = max_tokens fixa a saída; a latência termina no último evento de token. Em calculate_metrics(), TPOT usa N - 1, apesar da simplificação no texto do artigo. Copiamos essa política apenas para velocidade, preservando a execução local e sequencial.

Validação disponível: 17 testes sem GPU passaram, incluindo contagem divergente, um token e EOS no limite. Não houve execução real dos quatro modelos. A revisão acrescenta tamanho fixo apenas ao teste de velocidade. Não migra a inferência para vLLM.